

# Sentiment Analysis Using LSTM

**Student Name:** Hajra Khan

**Tool Used:** Python, TensorFlow, VS Code

## 1. Introduction

Sentiment Analysis is a Natural Language Processing (NLP) task that identifies whether a text expresses a positive or negative opinion. It is widely used in real-world applications such as product reviews, social media analysis, and customer feedback systems.

Text data is sequential in nature, meaning the order of words matters. Traditional machine learning models cannot properly understand word sequence and context. Recurrent Neural Networks (RNN) and Long Short-Term Memory (LSTM) networks are designed to process sequential data effectively.

In this project, an LSTM-based deep learning model was developed to classify IMDB movie reviews as either positive or negative.

## 2. Objective of the Project

The main objectives of this project were:

- To understand sequential data modeling.
- To implement an LSTM-based deep learning model.
- To preprocess text data using tokenization and padding.
- To train and evaluate model performance.
- To display sample predictions.
- To analyze accuracy and loss graphs.

## 3. Dataset Description

The IMDB movie review dataset was used for this project.

### Dataset Details:

- Total Reviews: 50,000
- Training Data: 25,000 reviews
- Testing Data: 25,000 reviews
- Classification Type: Binary (Positive / Negative)

Each review is converted into a sequence of numbers where each number represents a word index.

Labels:

- 0 → Negative Review
- 1 → Positive Review

## **4. Data Preprocessing**

Before training the model, several preprocessing steps were performed.

### **4.1 Vocabulary Limitation**

Only the top 10,000 most frequent words were selected.

This helps reduce model complexity and improves performance.

```
vocab_size = 10000
```

### **4.2 Tokenization**

Each review was already tokenized in numeric format by the IMDB dataset.

Example:

Instead of words, the review appears as a list of numbers.

### **4.3 Padding**

Reviews have different lengths. Neural networks require fixed-size input.

Therefore, padding was applied to make every review length equal to 200 words.

```
max_length = 200
```

After padding, the training data shape became:

```
(25000, 200)
```

This means:

- 25,000 reviews
- Each review contains 200 words

## **5. Model Architecture**

The model was built using TensorFlow and Keras Sequential API.

**Final Model Architecture:**

1. **Embedding Layer**

- Input dimension: 10,000
- Output dimension: 128
- Converts word indices into dense vectors.

## **2. First LSTM Layer**

- 64 units
- `return_sequences=True`
- Learns sequential patterns.

## **3. Dropout Layer**

- Dropout rate: 0.3
- Prevents overfitting by randomly disabling neurons during training.

## **4. Second LSTM Layer**

- 32 units
- Further refines sequence understanding.

## **5. Dense Output Layer**

- 1 neuron
- Sigmoid activation function
- Produces output between 0 and 1.

## **Model Compilation**

- Optimizer: Adam
- Loss Function: Binary Crossentropy
- Evaluation Metric: Accuracy

```
model.summary()
✓ 0.1s

Model: "sequential_1"


```

| Layer (type)            | Output Shape | Param #     |
|-------------------------|--------------|-------------|
| embedding_1 (Embedding) | ?            | 0 (unbuilt) |
| lstm_1 (LSTM)           | ?            | 0 (unbuilt) |
| dropout (Dropout)       | ?            | 0           |
| lstm_2 (LSTM)           | ?            | 0 (unbuilt) |
| dense_1 (Dense)         | ?            | 0 (unbuilt) |

```


Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)
```

## 6. Model Training

The model was trained with the following settings:

- Epochs: 5
- Batch Size: 64
- Validation Split: 20%

During training:

- Accuracy increased gradually.
- Loss decreased over epochs.
- Validation accuracy remained stable.

This shows that the model successfully learned from sequential data.

## 7. Model Evaluation

After training, the model was evaluated on the test dataset.

**Final Test Accuracy:**

**84.18%**

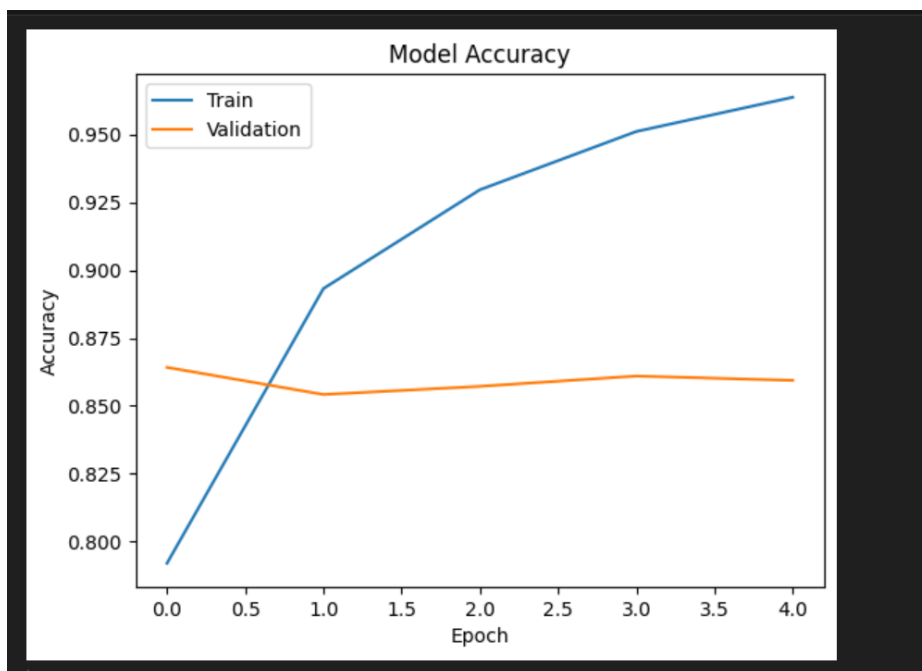
This means the model correctly predicted sentiment for approximately 84 out of 100 reviews.

## 8. Accuracy and Loss Graphs

Two graphs were generated:

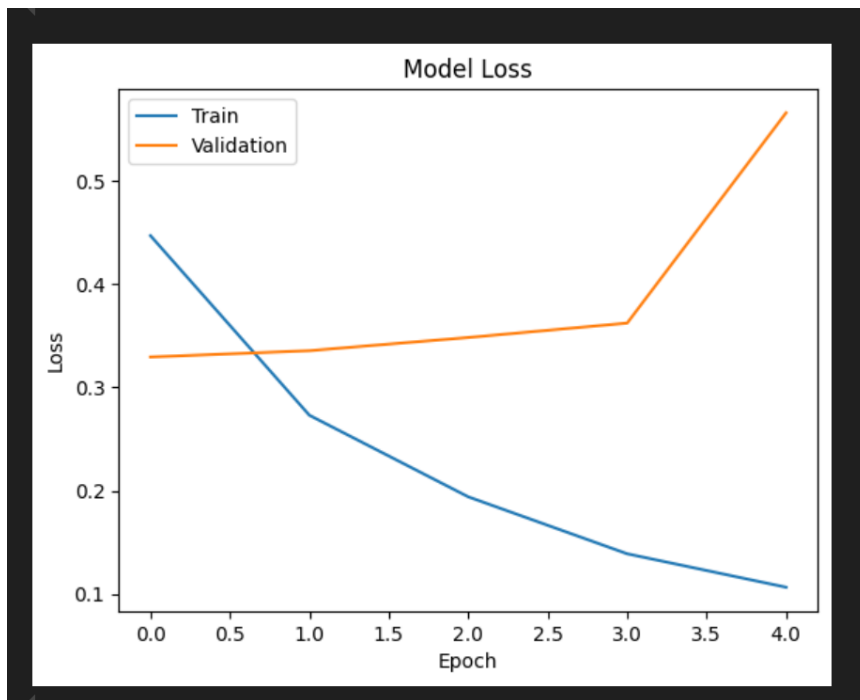
### 8.1 Accuracy Graph

- Training accuracy increases with each epoch.
- Validation accuracy follows a similar pattern.
- No major overfitting observed.



### 8.2 Loss Graph

- Training loss decreases over time.
- Validation loss stabilizes.
- Indicates good learning performance.



## 9. Sample Predictions

The model was tested on 5 sample reviews.

For each review:

- Model predicted Positive or Negative.
- Compared with actual label.

Example Output:

Review 1

Predicted: Positive

Actual: Positive

Review 2

Predicted: Negative

Actual: Negative

```
1/1 ————— 1s 558ms/step
Review 1
Predicted: Positive
Actual: Negative
-----
1/1 ————— 0s 88ms/step
Review 2
Predicted: Positive
Actual: Positive
-----
1/1 ————— 0s 88ms/step
Review 3
Predicted: Negative
Actual: Positive
-----
1/1 ————— 0s 118ms/step
Review 4
Predicted: Positive
Actual: Negative
-----
1/1 ————— 0s 98ms/step
Review 5
Predicted: Positive
Actual: Positive
```

The model correctly predicted most of the sample reviews.

## 10. Understanding Sequential Modeling

LSTM networks are designed for sequential data such as text.

Unlike traditional neural networks:

- LSTM remembers previous words.
- It captures long-term dependencies.
- It solves the vanishing gradient problem of basic RNN.

This makes LSTM highly suitable for sentiment analysis tasks.

## 11. Challenges Faced

- Python and TensorFlow installation issues.
- Kernel mismatch in VS Code.
- Training time management.
- Model tuning for better accuracy.

These challenges were solved during implementation.

## **12. Conclusion**

This project successfully implemented a Sentiment Analysis model using LSTM.

The model:

- Learned sequential patterns in text.
- Achieved 84.18% test accuracy.
- Demonstrated stable training performance.
- Successfully classified positive and negative reviews.

The project shows how deep learning and LSTM networks can effectively process and analyze textual data.

## **13. Future Improvements**

The model can be improved by:

- Increasing number of epochs.
- Using Bidirectional LSTM.
- Using pretrained word embeddings (GloVe).
- Hyperparameter tuning.
- Increasing sequence length.

These improvements may increase accuracy further.